



Comparison of Face Recognition Algorithms for Face Blurring in Videos

Alessandro Sartore, Sebastiano Tocci



Index

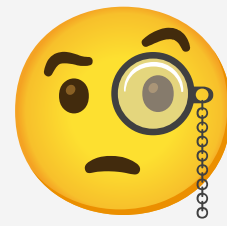
- [Introduction](#)
- [Problem](#)
- [Viola-Jones](#)
- [YOLO](#)
- [Blurring Algorithms](#)
- [Testing](#)
- [Conclusions](#)



Introduction

- ◆ **Why Face Blurring?**
 - Privacy protection in surveillance, media, GDPR compliance
- ◆ **What We Compare?**
 - Viola-Jones **VS** YOLO
 - Differences in speed, accuracy, robustness
 - Gaussian **VS** Median **VS** Pixelation
- ◆ **Our Approach**
 - WIDER FACE: A Face Detection Benchmark





Problem

2 separate subproblems:

- detecting faces
- applying the blur

Step #1 **much harder** than #2, especially in real time video streams!

– Need to balance detection performance and speed

• Key Challenges include:

- Varying lighting, occlusions, & orientations
- balancing False positives vs. false negatives



Viola - Jones approach: the Haar cascade classifier

Step 1: Integral image computation

Converts the input image into an “integral image” to quickly sum pixel intensities over rectangular regions, enabling fast Haar features computations

Step 2: Haar feature extraction

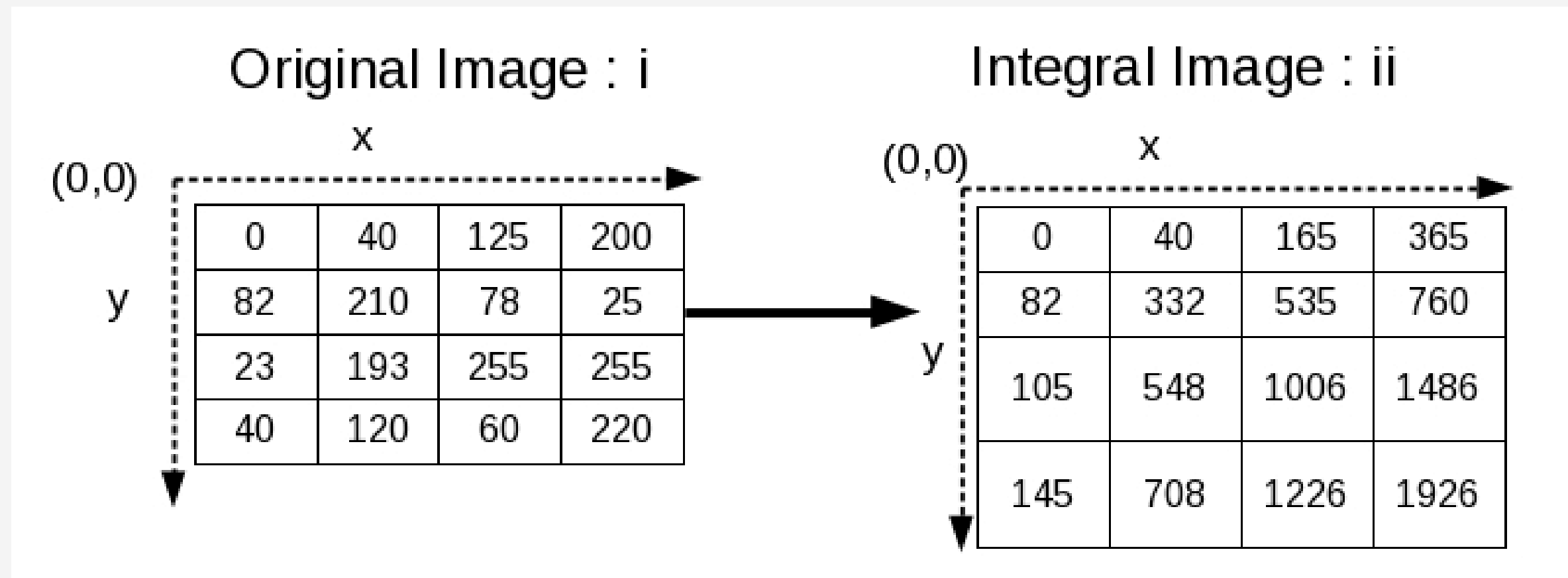
Using the integral image, computes simple rectangular features (known as Haar like features)

Step 3: Cascade classifiers

Applies a series of increasingly complex stages to reject non-face regions early, where each stage refines detection to reduce false positives and improve accuracy

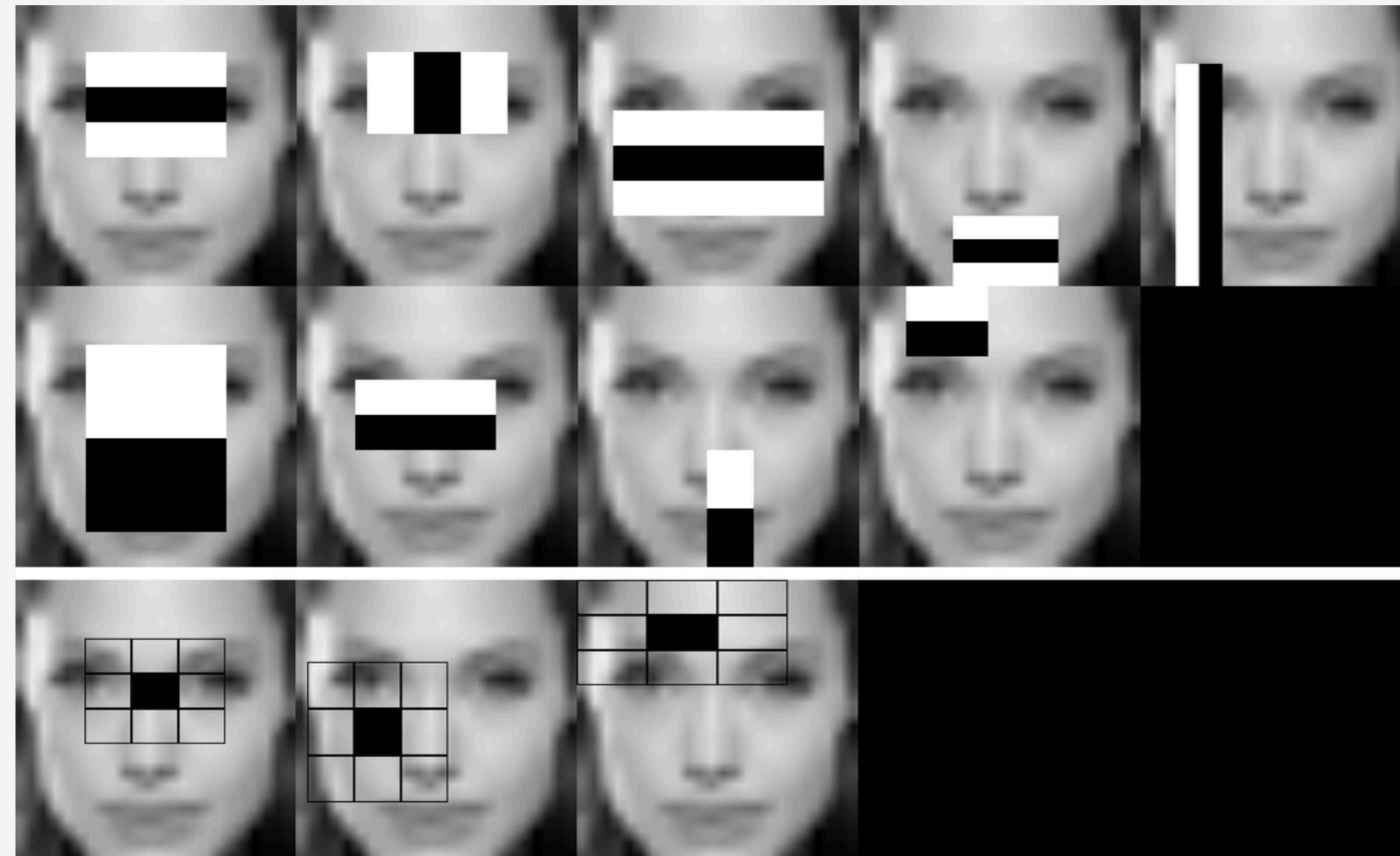
Step 1: Integral image computation

Converts the input image into an “integral image” to quickly sum pixel intensities over rectangular regions, enabling fast Haar features computations



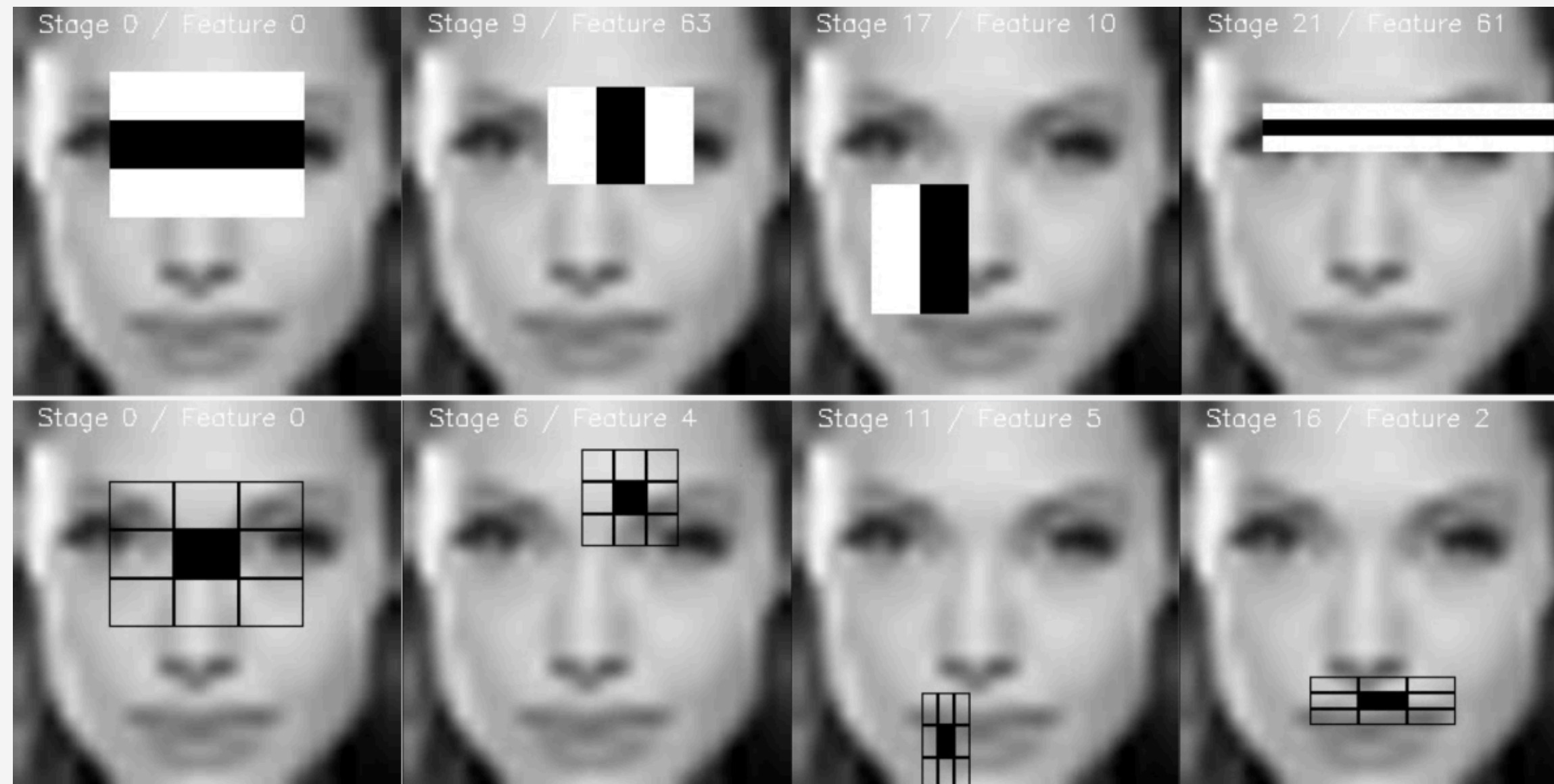
Step 2: Haar feature extraction

Using the integral image, computes simple rectangular features (known as Haar like features)



Step 3: Cascade classifiers

Applies a series of increasingly complex stages to reject non-face regions early, where each stage refines detection to reduce false positives and improve accuracy





Viola - Jones: performance considerations

Main strengths

⚡ **Speed:** The cascade of classifiers allows to quickly discard negative regions; combined with the fact that there are no deep neural networks, the inference is overall very cheap!

Main weaknesses

!/? **Accuracy:** the model struggles in more challenging conditions, for instance when the faces are partially occluded, they are not well lit or they are in unusual positions (i.e. turned on one side instead of front facing)



YOLO: You Only Look Once

Step 1: Image Processing & Grid Division

- 📌 The input image is divided into a grid (e.g., 13×13 for YOLOv3).
 - 📌 Each grid cell is responsible for detecting objects within its area.
-

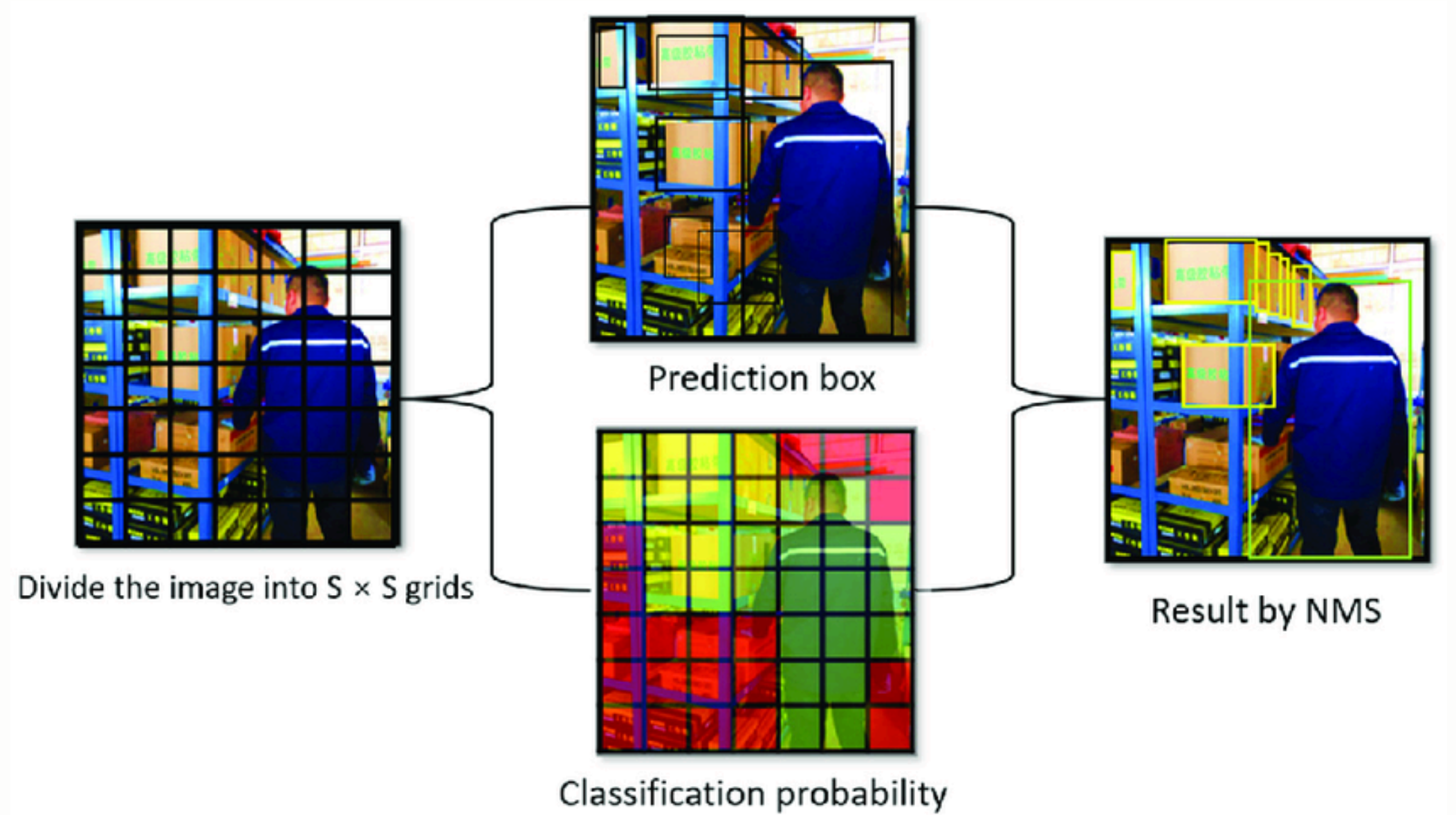
Step 2: Bounding Box Prediction

- 📌 Each grid cell predicts multiple bounding boxes with confidence scores.
 - 📌 Each grid cell also predicts class probabilities for the detected objects.
-

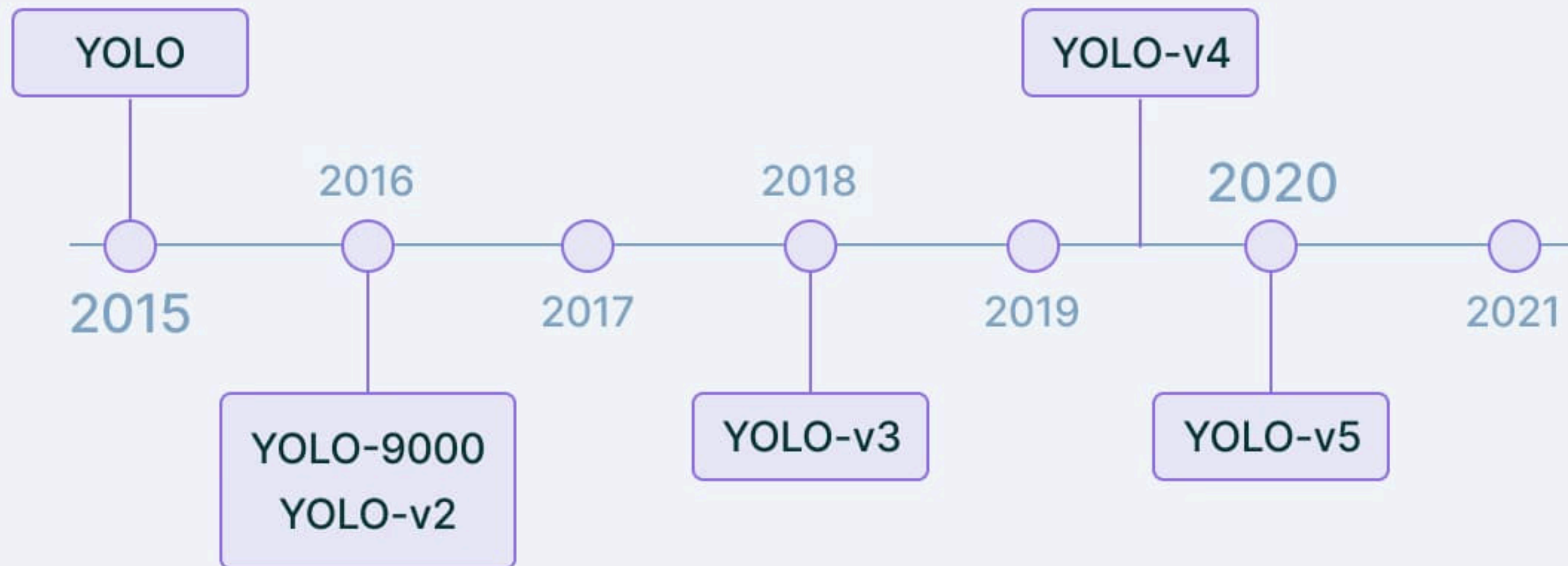
Step 3: Non-Maximum Suppression (NMS)

- 📌 Filters overlapping boxes, keeping only the most confident detections.
- 📌 Outputs final detected faces with bounding boxes & confidence scores.

YOLO: You Only Look Once



YOLO timeline



V7 Labs

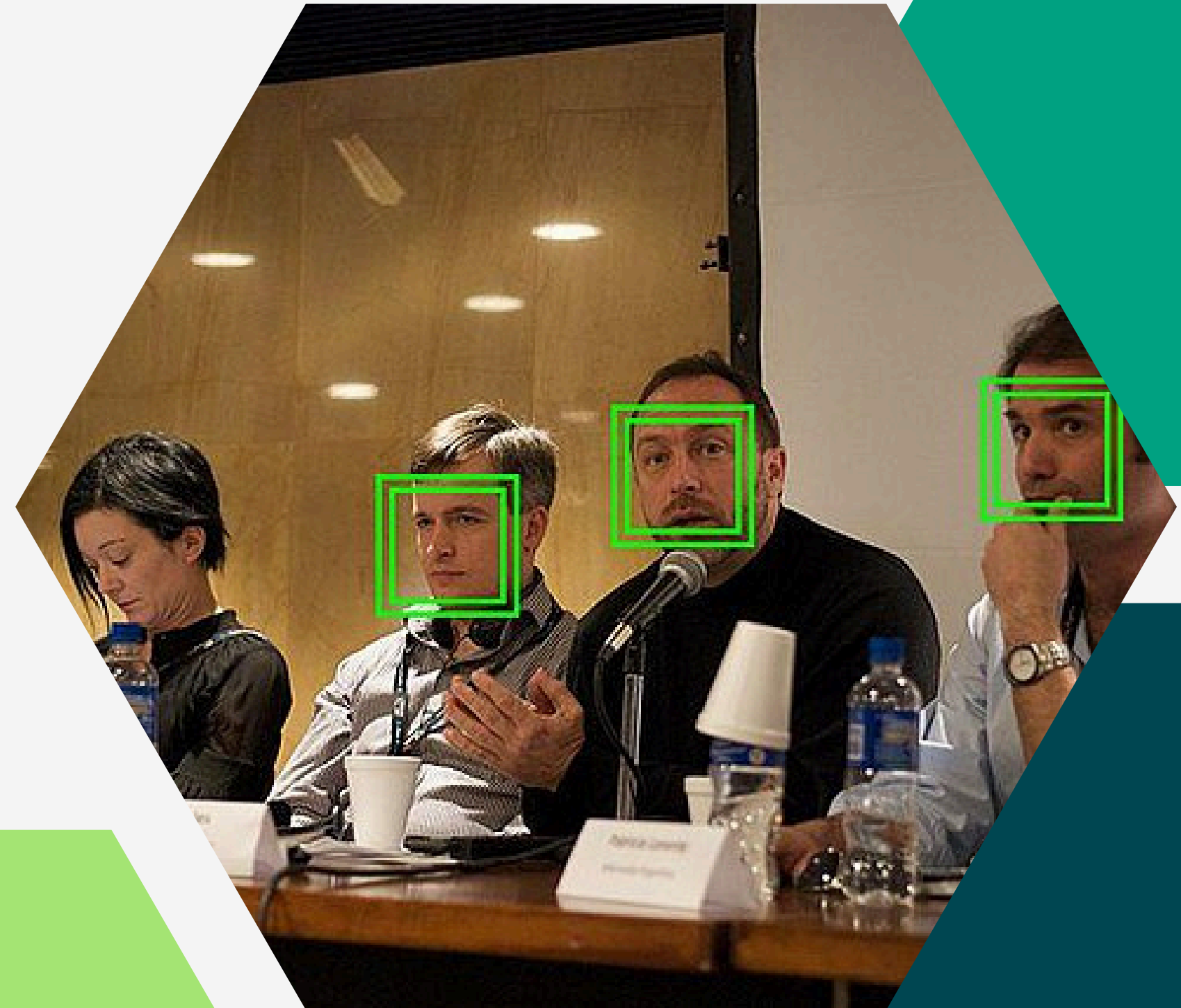
⚡ YOLO: Strengths & Weaknesses

✓ Strengths

- ✓ High Accuracy & Robustness – Works well under different lighting & angles
- ✓ Multi-Face Detection – Identifies multiple faces in a single pass
- ✓ Faster than CNN-based detectors – More efficient than traditional deep learning models

✗ Weaknesses

- ✗ Computationally Expensive – Requires GPUs for real-time performance
- ✗ Struggles with Small Faces – Grid-based predictions may miss very small faces



Face Blurring Techniques

Gaussian Blur (Smooth & Natural)



🧠 Applies a Gaussian filter to **soften facial details**.

🖼️ **Preserves** some **shape** but removes sharp edges & fine details.

⚡ Effective for **natural-looking anonymization**.

Median Blur (Extreme & Distorted)



🖼️ Replaces pixels with the **median value of surrounding pixels**.

🧠 Creates an **intense distortion** effect, making **recognition** nearly **impossible**.

⚡ Best for **high-security anonymization**.

Pixelation (Blocky & Recognizable)



🧠 **Reduces resolution** by dividing the face into larger square pixels.

⚡ **Faster to compute** but may still allow identity recognition at low blur levels.

🖼️ Useful for **moderate privacy** needs where **some details** can remain **visible**.



Self training Viola Jones

Given the relative simplicity of the model, we decided to try and train it ourselves.

Unfortunately, it wasn't as simple as it appeared!

- We had to resurrect a very old version of OpenCV (the 3.4, from 2017), as cascade classifiers have been deprecated ever since
- Had to compile it from source, but dependencies required outdated version (i.e. FFmpeg), so we disabled them





Self training Viola Jones (cont)

Once we had the tools compiled, we downloaded the WIDER faces dataset, we updated the bounding box file to match OpenCV's desired format and we started the training.

```
opencv_traincascade -data cascade-classifier -vec training-samples  
-bg non-faces.txt -precalcValBufSize 10000 -precalcIdxBufSize 10000  
-numThreads 18 -acceptanceRatioBreakValue 0.00001 -numPos 2000 -numNeg 1000
```





Benchmarking & Performance Evaluation

Evaluation Criteria

-  How well do they detect faces? (Precision, Recall, IoU)
-  Speed: How fast do they process images & videos? (Processing Time)

Resource Usage

Tested on

- Dataset: **WIDER FACE** 
- Tool Used: **Runlim** 





Experimental Setup

Dataset: WIDER FACE

- Includes **images of faces** with varying difficulties.
 - Used **validation & test sets** for evaluation.
-

Benchmarking Tool: Runlim

- Measures **execution time & memory usage** for fair comparison.
-

Hardware Used

- **CPU:** Intel Core i7-8550U @ 3.60GHz
 - **RAM:** 16GB
 - **GPU:** NVIDIA GeForce 940MX (for YOLO)
-

Key Metrics Evaluated

- 1 **Precision & Recall** – Measures accuracy of detections.
- 2 **Intersection over Union (IoU)** – Evaluates bounding box alignment.
- 3 **Processing Time** – Checks speed of detection.



Viola-Jones Comparison: Self vs. Pre Trained

Algorithm	Scale	MinNeighbour	Total Faces	Detected	True Positive	False Positive	False Negative
Viola-Jones (PreTrained)	1.1	5	39708	7696	5116	2580	34592
Viola-Jones (PreTrained)	1.07	4	39708	11505	5803	5702	33905
Viola-Jones (PreTrained)	1.05	3	39708	18406	6499	11907	33209
Viola-Jones (PreTrained)	1.15	3	39708	8965	5377	3588	34331
Algorithm	Scale	MinNeighbour	Total Faces	Detected	True Positive	False Positive	False Negative
Viola-Jones (SelfTrained)	1.1	5	39708	47555	3063	44492	36645
Viola-Jones (SelfTrained)	1.07	4	39708	73439	4036	69403	35672
Viola-Jones (SelfTrained)	1.15	3	39708	61988	3474	58514	36234

Viola-Jones Comparison: Self vs. Pre Trained

Algorithm	Scale	MinNeighbour	Precision	Recall	Average IoU	Time (m)	Space (MB)
Viola-Jones (PreTrained)	1.1	5	0.6648	0.1288	0.6873	8.58	326.8
Viola-Jones (PreTrained)	1.07	4	0.5044	0.1461	0.6848	12.16	404.7
Viola-Jones (PreTrained)	1.05	3	0.3531	0.1637	0.6817	17.08	511.3
Viola-Jones (PreTrained)	1.15	3	0.5998	0.1354	0.6868	6.46	275.8
Algorithm	Scale	MinNeighbour	Precision	Recall	Average IoU	Time (m)	Space (MB)
Viola-Jones (SelfTrained)	1.1	5	0.0644	0.0771	0.6839	19.28	328.4
Viola-Jones (SelfTrained)	1.05	4	0.0550	0.1016	0.6798	25.29	404.4
Viola-Jones (SelfTrained)	1.15	3	0.0560	0.0875	0.6819	14.9	273.7



Viola-Jones vs. YOLO: Performance Comparison

Algorithm	Scale	MinNeighbour	Total Faces	Detected	True Positive	False Positive	False Negative
Viola-Jones (PreTrained)	1.1	5	39708	7696	5116	2580	34592
Viola-Jones (PreTrained)	1.07	4	39708	11505	5803	5702	33905
Viola-Jones (PreTrained)	1.05	3	39708	18406	6499	11907	33209
Viola-Jones (PreTrained)	1.15	3	39708	8965	5377	3588	34331
Algorithm	NMS	Confidence	Total Faces	Detected	True Positive	False Positive	False Negative
YOLO	0.4	0.5	39708	23383	18452	4931	21256
YOLO	0.5	0.4	39708	27227	19706	7521	20002
YOLO	0.6	0.6	39708	21560	18203	3357	21505



Viola-Jones vs. YOLO: Performance Comparison

Algorithm	Scale	MinNeighbour	Precision	Recall	Average IoU	Time (m)	Space (MB)
Viola-Jones (PreTrained)	1.1	5	0.6648	0.1288	0.6873	8.58	326.8
Viola-Jones (PreTrained)	1.07	4	0.5044	0.1461	0.6848	12.16	404.7
Viola-Jones (PreTrained)	1.05	3	0.3531	0.1637	0.6817	17.08	511.3
Viola-Jones (PreTrained)	1.15	3	0.5998	0.1354	0.6868	6.46	275.8
Algorithm	NMS	Confidence	Precision	Recall	Average IoU	Time (m)	Space (MB)
YOLO	0.4	0.5	0.7891	0.4647	0.7306	22.47	1824.3
YOLO	0.5	0.4	0.7238	0.4963	0.7244	22.67	1823.1
YOLO	0.6	0.6	0.8443	0.4584	0.7322	22.70	1823.7



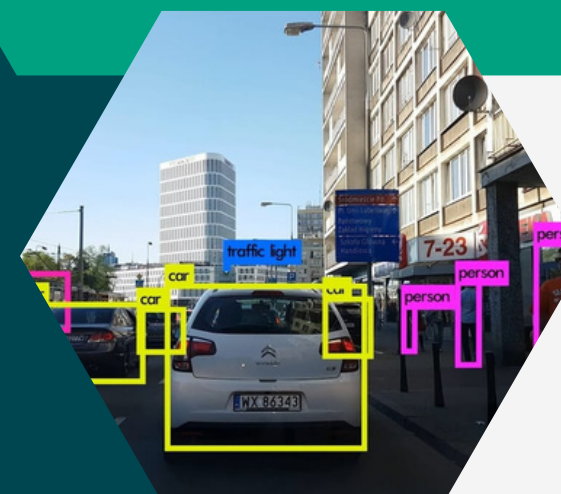
Speed vs. Accuracy: The Trade-off



◆ Viola-Jones

✓ **Very fast** – Works well for real-time applications

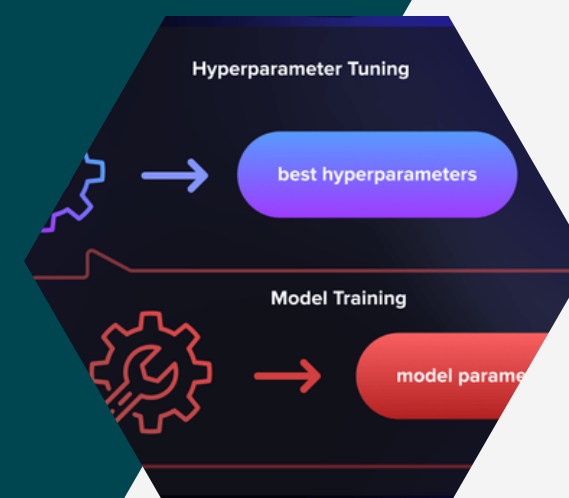
✗ **High false negatives** – Misses many faces in complex scenarios



◆ YOLO

✓ **High accuracy** – Detects faces even in difficult conditions

✗ **Computationally expensive** – Requires GPU for real-time performance



📌 Tuning Parameters → Impact on Results

Viola-Jones: Lower Scale Factor → Higher recall, more false positives

YOLO: Lower Confidence Threshold → Higher recall, but more false positives

Best Use Cases for Each Algorithm

Viola-Jones	YOLO
✓ Real-time applications – Low computational cost, fast execution ⚡	✓ High-accuracy face detection – Detects faces in various conditions 🎯
✓ Simple & controlled environments – Well-lit, frontal faces 📷	✓ Multiple faces in a scene – Better handling of occlusions & lighting 🔦
✓ Low-power devices – Works on CPUs, no GPU required 💻	✓ Post-processing on videos – Ideal for privacy enforcement 🔍



Video Testing Results



Test Videos Used

- ✓ **test.mp4** – Simple, well-lit single face ✓
- ✓ **test2.mp4** – Multiple faces, varied lighting, angles 🤔
- ✓ **test3.mp4** – Low lighting, occlusions, challenging conditions ●



Performance Observations

- **test.mp4**: Both Viola-Jones & YOLO performed well, but **Viola-Jones** was faster
- **test2.mp4**: **YOLO outperformed** as Viola-Jones missed many faces
- **test3.mp4**: **Viola-Jones failed** to detect faces, while **YOLO** still worked with **minor inaccuracies**



Conclusion

- Viola-Jones is fast but unreliable in complex videos
- YOLO is slower but robust in real-world scenarios



Conclusion

- ◆ YOLO outperforms Viola-Jones but at much higher computational cost
- ◆ The choice therefore is non trivial, it depends on the specific use-case
- ◆ Similarly, different blurring techniques require different computational resources. The tradeoff is between privacy preservation and speed





Thanks for your attention! Questions?

Alessandro Sartore, Sebastiano Tocci

